

End-to-end test report - 14 September 2026

Every number in this folder came from a running system exercised over HTTP or from a container that was actually stopped, killed or restarted. Nothing here is from the unit or integration suites.

Read in order: this summary, then the numbered files for the detail.

e2e-report-2026-09-14.pdf in this folder is the same content as one document, for a reader outside the repository. It is generated by `scripts/build-e2e-report.py` from the Markdown beside it, so it can be rebuilt rather than being a copy that drifts.

File | What it holds

01-storefront-contract.md | filtering, pagination, sorting by effective price, validation edges

02-promotions-live.md | create, assign, cancel, the conflict rule, the pricing floor

03-ingestion-live.md | Scenario A through the HTTP route, and its two durations

04-recovery-and-load.md | resumability under a kill, the memory floor, read load

05-scenario-b.md | the flash sale under load, and the measurement that could falsify it

06-edge-cases.md | a window shorter than the sweep, and the validation surface

07-failure-injection.md | Redis and PostgreSQL restarted under load, the racing writers

08-drift-and-retry.md | the read model's drift repaired from a real injection, and bulk retry

The case study's questions, answered with measurements

List products - filtering by category, pagination, and sorting by effective price. All three live. Sorting is by the `_effective_price`: a 60 %-discounted 200.00 product sorts ahead of a 100.00 one, because the shopper pays less for it.

"Get single product details (the most highly trafficked endpoint)."

4 375 req/s at p50 22 ms and p99 35 ms, 100 connections - the fastest surface in the system, as it should be. It is one hash read where the listing is a sorted-set range plus a multi-get.

"Create, cancel, and assign promotions." All three, with cancel idempotent and a missing promotion answering 404.

"A product can have at most one active promotion at a time." Enforced per target at write time by a GiST exclusion constraint - ****eight concurrent writers on one product produce one 201 and seven 409**** - and resolved at read time, where a product's own 70 % beats a category's 50 % because the shopper gets the better of the two.

Scenario A - 500 000 rows under serverless constraints

POST `/api/vendor/imports` | 202 in **834 ms**, including streaming and hashing 22 MB

Chunks | 6, each on its first attempt

Rows | 500 000 processed, **0 rejected**

Worker memory | **flat at 51.6-54.9 MiB** across all six chunks, against a 256 MiB limit

Flat is the property, not the peak - a file six times larger draws the same

line, because what is resident is one batch rather than one file.

Killed mid-import with SIGKILL, the chunk resumed from its checkpoint: the killed chunk returned **87 783 rows, not 1 000 + 87 783**, and the six chunks summed to **exactly 500 000**. The arithmetic is the proof; it would have read 501 000 the other way.

The memory floor is between 32 and 48 MiB. A 48 MiB container with a 32 MiB heap completed a 100 000-row import in 11 s; a 32 MiB container was OOM-killed at startup before claiming anything. ****What fails first is the Node runtime's own footprint, not the batch**** - the import path runs in under a quarter of what the case study allows. And it failed `_safely_`: nothing stored, nothing reported as success, and a normally sized worker later completed the same import unprompted.

Scenario B - a flash sale on 100 000 products

Twice the case study's 50 000.

The measurement that could have falsified the design, reported first: a read-only burst of **34 400 requests moved PostgreSQL by ten transactions** - and those ten are the readiness probe and the workers. The idle stack commits about one a second on its own. **The read path does not reach PostgreSQL.** Had that delta tracked the request count, the read model would have been a cache with a fallback.

req/s | p50 | p99 | non-2xx | errors

quiet | 1 721 | 28 ms | 42 ms | 0 | 0

during the sale's recompute | 1 280 | 32 ms | 104 ms | 0 | 0

during a 500 000-row import | 3 614 (detail route) | 26 ms | 51 ms | 0 | 0

Propagation from the 201: ****first product discounted at 915 ms, all 100 000 at 12.2 s****; cancel reverts the first at 993 ms and all 100 000 at 10.6 s.

****A new product added to the category mid-sale is served at the sale price 444 ms later****, with no further request - the case study's own sentence, through the real queues.

So the bottleneck question, answered as a measurement: ****the writes never block the reads, because the reads never reach the store the writes contend for.**** A sale costs a quarter to two fifths of the throughput and doubles to triples the tail while it runs; an import costs about a sixth and 16 ms. Both are felt as latency, neither as failure.

What broke, and what it cost

****One defect, found by restarting PostgreSQL under load and fixed the same night (#128).**** The recompute survived - 100 001 of 100 001 came back - but the api process died, because `createPool` attached no error listener and node-postgres documents that an idle client error with no listener is fatal. ****A database restart took the HTTP listener with it, so storefront reads that never touch PostgreSQL**

failed for its duration** - the design's central claim failing in the one place it can, through an unhandled event rather than a query.

Redis restarted mid-recompute is a pass: 13 x 503 across about six and a half seconds, no stale price, no fallback, everything correct afterwards. The reconciler's five-minute chain survived the same restart and its next sweep fired on schedule.

Known gaps, named here rather than left to be found

1. **A lost announcement is repaired after the watermark reaches it, not within one reconciler period.** The sweep advances its watermark at most an hour per run, so on a database whose watermark is behind, repair latency is the catch-up time. Measured: a destroyed announcement was repaired about fifteen minutes later, and only because that promotion's own `starts_at` fell in the window being caught up through.
2. **A chunk that exhausts its attempts stays running with an expired lease** and nothing reclaims it; the vendor is then refused by the one-import-per-vendor index. The orphan-chunk sweep that would reclaim it does not exist.
3. **A duplicate SKU inside one file is counted in neither `rowsProcessed` nor `rowsRejected`**, so the vendor's arithmetic does not close. ON CONFLICT cannot affect the same row twice, so the dedupe is forced by the mechanism; a third counter is the honest fix, since a row that lost to a later row was not rejected.
4. **One vendor's import silently replaces another's product** for the same SKU, and **a third party can permanently lock a vendor out** by registering their file first. Both are the missing vendor column; ADR-0005 records `unique (vendor, file_sha256)` as the fix and names the scope as the owner's to set.
5. **Nothing authenticates any route.** vendor is a form field, and `GET /api/vendor/imports/:id` returns any import by id.

How to read the numbers

Every file ends with a **Vantage** section. That is not ceremony: the day's most repeated defect was a figure that answered a narrower question than the sentence above it, and two of them were caught in this campaign - a schema comparison whose query errored on both sides and reported "identical", and a transaction delta that came back **negative** because the counters had been reset underneath it.

* **Memory is container accounting**, never host RSS. The two are different quantities and a host sampler cannot see a container boundary.

* **Latencies are loopback**, with no network between client and server, so they are a ceiling for this machine rather than a service level.

* **Durations name their system.** The 29 s on record for a 500 000-row import is

the importer alone with nothing consuming; 128 s is the whole system keeping its read model current. Neither may be quoted without saying which.

* Where two runs of one thing disagree - 1 280 against 1 013 req/s under a sale - both are reported. Picking one would be the more confident and less honest report.

Storefront contract, probed live

Run against the compose stack on 127.0.0.1:3100, built from main with the monitoring profile up. Every figure below came from a real HTTP request; nothing here is from the test suite.

Catalogue at the time of the run: **100 304 products**, 20 000 of them in Accessories.

The case study's three core expectations

"List products (Must include filtering by category, pagination, and sorting by effective price)."

Probe | Result

?sort=effectivePrice&order=asc&pageSize=3 | 200, cheapest first - 1051, 1051, 1051 cents

?sort=effectivePrice&order=desc&pageSize=3 | 200, dearest first - 93 702 cents (LIVE - *, the markup chain)

?category=Accessories | 200, total 20 000, every item in the category

?page=1 vs ?page=2 at pageSize=2 | disjoint id sets, page echoed

GET /api/products/:id | 200 with effectivePriceCents and promotion

Sorting is by the effective price, not the base price - confirmed separately by the review session on a controlled pair: Alpha at 100.00 and Bravo at 200.00 with a 60 % promotion on Bravo alone sorts **Bravo first ascending** at 8 000 against Alpha's 10 000. The dearer product sorts first because the shopper pays less for it.

Validation edges

Every one of these answered 400 with the project's error envelope rather than a stack trace or a silent default:

page=502&pageSize=20 (offset 10 020, over the 10 000 cap) . pageSize=101 .

pageSize=0 . sort=basePrice (the only sort is effectivePrice) .

order=sideways . a 257-character category . page=-1 . page=abc .

unknown=1 (an unknown query field is refused rather than dropped).

page=501&pageSize=20 - offset exactly 10 000 - answers 200. The boundary is inclusive and the refusal starts one page later.

Path parameter: 0, -1, abc, 1.5 and 1 OR 1=1 all answer 400;

99999999 answers 404. A malformed id never reaches the bigint column.

Scenario B's functional sentence

"if a brand new product is added to the 'Accessories' category while the flash sale is active, that product must automatically benefit from the discount"

Verified through the real queues, not in process: with a 50 % category sale running, POST /api/products created a product at 300.00 and the storefront served it at **150.00 naming the sale 444 ms later**, with no further request.

The same listing shows the resolver's precedence in the live system: a product carrying its own 60 % keeps it during a 50 % category sale, because 8 000 beats 10 000. **The shopper gets the better of the two, not the more specific one.**

Vantage

Single machine, containers on one host, catalogue as stated. These are **functional** results and one propagation latency; they are not load figures and must not be quoted as service levels.

Promotions: create, assign, cancel, conflict - probed live

Real HTTP against the compose stack on 127.0.0.1:3100. A private category was used so nothing here disturbed the 100 000-product catalogue the load runs use.

The case study's third core expectation

"Create, cancel, and assign promotions to products or categories."

Probe | Result

POST /api/promotions with category | 201, status: active, state: live

POST /api/promotions with productId | 201

POST /api/promotions/:id/cancel | 200, status: cancelled

cancel the same promotion again | 200, same body - idempotent, not a 409

cancel an id that does not exist | 404

"A product can have at most one active promotion at a time"

A second promotion overlapping the same category window is refused:

```
409 {"error":{"message":"An active promotion already covers that target for this window"}}
```

The refusal names the target and the window and ****not** the promotion it collided with****** - the conflicting row is another record the caller never saw.

A product-level promotion is not a conflict with a category-level one, and this

is the interesting half. With a 50 % sale running on the category, a 70 %

promotion on one product was accepted (201), and the storefront then served:

```
EV-...-1 3000 "Ev prod ..." <- its own 70 %
EV-...-2 5000 "Ev sale ..."  <- the category's 50 %
```

So "one active promotion at a time" is enforced **per target** at write time, and the resolver decides which one applies at read time. Cancelling the product promotion returned that product to the category price (5000) without another request.

Pricing edges

* **A fixed discount larger than the base price floors at zero, not below.**

A 9 999.99 discount on a 100.00 product served `effectivePriceCents: 0`. The write was accepted - the floor is in the calculation, not the validator, which is the right place for it: the same promotion applied to a dearer product is meaningful.

* **Sorting handles the zero correctly**: ascending put the zero-priced product first.

Validation refusals, all 400 with the envelope

* a promotion whose whole window is in the past

* `endsAt` before `startsAt`

* a percentage above 100

Propagation

Each write above was visible in the storefront within the few seconds between the request and the next read. A measured figure for this exists separately: **444 ms** from a POST `/api/products` to the storefront serving it at the sale price, through the real queues.

Vantage

Single machine, one host, catalogue ~100 000 products. Functional results; no figure here is a load or latency measurement.

Ingestion: Scenario A through the HTTP route

Real HTTP against the compose stack. The 500 000-row run below was taken by the review session; the edge cases were taken separately. Nothing here came from the test suite.

Scenario A, the whole sentence

"500,000 rows ... every record must pass through internal dynamic pricing rules ... strict Timeout limit ... memory is highly restricted ... the processor halts as soon as the HTTP request completes (Stateless)."

POST /api/vendor/imports with a 22 MB, 500 000-row file:

Response | 202 { jobId, chunksTotal } in **834 ms**, including streaming 22 MB to disk and hashing it

Chunks | 6, each completed on its **first attempt**

Rows | 500 000 processed, **0 rejected**

Worker memory | **47.9 MiB at start, 51.6-54.9 MiB from the first chunk to the sixth**, against a 256 MiB container limit and a 192 MiB V8 ceiling

Flat is the property, not the peak. A file six times larger draws the same line, because what is resident is one batch rather than one file. That is the answer to "memory is highly restricted", and the request returning in 834 ms while six chunks run afterwards is the answer to "the processor halts as soon as the HTTP request completes".

99 999 rows were skipped by the is distinct from **guard** because they were byte-identical to an earlier import - the guard proving itself on a number nobody arranged.

Two durations that measure two different systems

The repository has **29 s** on record for a 500 000-row import. This run took **128 s**. The machine is not the difference.

* **29 s** is the importer alone, with nothing consuming the announcements.

* **128 s** is the whole system keeping its read model current: every batch the importer announces is re-priced against the same PostgreSQL while the next chunk is being read. The read-model consumer peaked at **154.8 MiB** doing it.

Both are true. Neither may be quoted without saying which one it is.

Edge cases

A hostile CSV - CRLF line endings, a quoted comma inside a field, no trailing newline, and the same SKU twice - answered 202 and completed. The line endings, the quoted comma and the missing final newline were all handled correctly.

One counting gap, and it is in the mechanism rather than the parser. Three data rows produced rowsProcessed: 2, rowsRejected: 0. ON CONFLICT DO UPDATE cannot affect the same row twice in one statement, so a duplicate SKU inside a batch is

deduped before the upsert and rowsProcessed is the count of distinct SKUs returned, not rows read. The vendor's arithmetic therefore does not close, and nothing names the difference.

Folding those rows into rowsRejected would be a different wrong number: **a row that lost to a later row for the same SKU was not rejected.** The honest fix is a third counter. Recorded, not taken.

A product changing category through a re-import leaves the old listing.

MOVE-X imported into CatOne (total 1), re-imported into CatTwo: CatOne total 0, CatTwo total 1. Sorted-set membership follows the move. There is no product update endpoint - PATCH /api/products/:id is 404 - so a category move is reachable only through ingestion.

Findings carried from the uniqueness model

* **One vendor can permanently refuse another vendor's file.** file_sha256 is unique globally rather than per vendor, so byte-identical files collide across vendors and the second is refused for ever. ADR-0005 records `unique (vendor, file\_sha256)` as the fix and names the key's scope as the owner's to set.

* **products.sku is globally unique with no vendor column**, so a later import overwrites another vendor's rows for the same SKU.

Neither is an endpoint defect; both are the uniqueness model.

Vantage

Single machine, containers on one host, PostgreSQL 16. Container accounting for memory (not host RSS). Durations are wall time for the whole system unless the line says otherwise.

Recovery, resumability and read load

Real HTTP and real containers against the compose stack. Nothing here is from the test suite.

Recovery when the read-model consumer is down

Stopped event - handler, cancelled a promotion, restarted it.

Step | Storefront

promotion live | 15 000 cents, naming the sale

consumer stopped, promotion cancelled | **still 15 000, still naming a cancelled promotion**

consumer restarted | **30 000, no promotion**, unprompted

Stale while the consumer is down is correct - the announcement was queued and undelivered, not lost. The queue is what makes the recovery automatic.

Recovery when the announcement is genuinely lost

Same setup, then DEL bull:promotions:wait while the consumer was down, so only the reconciler's boundary sweep can repair it.

****The repair did not arrive within a reconciler period, and the reason is worth more than the fact.**** The sweep is running on schedule and repairing nothing:

```
since 2026-09-13T20:43 windowEnd 2026-09-13T21:43 repaired 0 "boundary sweep complete"
```

while wall time is **03:07 on the 14th**. The watermark was seeded when the database was migrated, and the sweep advances it by at most an hour per run (MAX_WINDOW_SECONDS). At a five-minute schedule that is twelve hours of history per wall-clock hour, so the window reaches a cancellation made now roughly half an hour later.

This is the hour clamp working exactly as ADR-0007 specifies - the clamp exists so a long outage cannot be read in one unbounded statement - but the ****observable consequence belongs in the evidence rather than in a trade-off paragraph****: on a database whose watermark is behind, a lost announcement is repaired after the watermark reaches it, not within one reconciler period. "Converges without operator action" is true; "within five minutes" is not, until the watermark has caught up.

Resumability under a kill (Scenario A's serverless constraint)

SIGKILL to the ingestion worker after two of six chunks, with the third checkpointed at 1 000 rows:

* the killed chunk returned with **attempts 2 and 87 783 rows** - that chunk's total, **not** 1 000 + 87 783;

* the six chunks sum to **exactly 500 000**;

* 500 000 products carry the job.

Had the resumed attempt restarted its chunk from the beginning, the sum would have

read 501 000. **The arithmetic is the proof**, and it would have been visibly wrong the other way.

`docker kill` is an operator stop, so `restart: unless-stopped` correctly did not restart the container; it was started by hand as a crash restart would.

Where the memory floor actually is

The same 100 000-row import on a throwaway worker against the same queue and stores:

Container | V8 heap cap | Result

96 MiB | 64 MiB | completed, 22-23 MB heap used

48 MiB | 32 MiB | completed in 11 s

32 MiB | 24 MiB | **OOM-killed at startup**, exit 137, before claiming any chunk

What fails first is the Node runtime's own footprint, not the batch. The floor is between 32 and 48 MiB, so the import path runs in ****under a quarter of the 256 MiB the case study allows****.

And it failed `_safely_`: with no worker able to start, the job stayed running with both chunks pending at zero attempts and nothing stored, and a normally sized worker later completed the same import with all 100 000 rows, unprompted. ****No job reported success having stored nothing.****

Read load, quiet stack

Catalogue ~100 000 products, containers on one host, no write traffic.

Endpoint | Connections | req/s | p50 | p99 | non-2xx | errors

GET /api/products?category=Accessories&pageSize=20 | 50 | **1 728** | 28 ms | 48 ms | 0 | 0

GET /api/products/:id | 100 | **4 375** | 22 ms | 35 ms | 0 | 0

The detail endpoint - which the case study calls "the most highly trafficked endpoint in the system" - is the faster of the two under twice the concurrency: it is a single hash read where the listing is a sorted-set range plus a multi-get.

Two places where recovery needs an operator today

1. **A watermark behind wall time delays every boundary repair** until it catches up (above).
2. **A chunk that exhausts its attempts stays running with an expired lease** and nothing reclaims it; the vendor is then refused by the one-import-per-vendor index. Observed on its own during the memory runs, not arranged. The orphan-chunk sweep that would reclaim it does not exist yet.

Vantage

Single machine, containers on one host, PostgreSQL 16, catalogue ~100 000 products. Container accounting for memory, never host RSS. Latencies are loopback with no network between client and server, so they are a **ceiling** for this machine

rather than a service level.

Scenario B: a flash sale under read load

"The second a campaign is created, it instantly affects over 50,000 products.

During flash sales, the GET /products listing endpoint (the storefront) experiences massive read traffic ... Develop a strategy to prevent the system from encountering bottlenecks during these simultaneous heavy read and write operations."

Real HTTP with autocannon against the compose stack. The sale below ran on Accessories holding **100 000 products - twice the case study's 50 000**.

The measurement that could have falsified the design

The design's claim is that storefront reads are served from Redis and ****do not reach PostgreSQL****. That is falsifiable, so it is reported first.

pg_stat_statements is not available on this stack - it needs a preload and a restart, and installing it would change the system being measured.

pg_stat_database answers the same question and is already there.

Window | Requests served | PostgreSQL transactions

idle, 2 s | 0 | 2

read-only burst, 20 s | **34 400** | **10** (and 217 tuples)

GET /api/products/:id, 10 s at 100 connections | 22 133 | 8

Ten transactions for thirty-four thousand storefront reads - and those ten are the readiness probe and the workers, not the reads. The idle stack commits about one a second on its own, so **the read load adds nothing above its own floor**.

Had that delta tracked the request count, the read model would have been a cache with a fallback and ADR-0003 would have been wrong.

The sale, created while the load was running

req/s | p50 | p99 | non-2xx | errors

quiet baseline | **1 721** | 28 ms | 42 ms | 0 | 0

during the sale (77 000 requests over 60 s) | **1 280** | 32 ms | 104 ms | **0** | **0**

during the cancel | **1 366** | - | 70 ms | **0** | **0**

Propagation, measured from the 201:

first product | all 100 000

sale applied | **915 ms** | **12.2 s**

cancel reverted | **993 ms** | **10.6 s**

****The sale costs about a quarter of the throughput and doubles the tail while it runs, and nothing once it has.**** Not one request failed, timed out or returned a non-2xx in either window.

The 205 000 tuples PostgreSQL returned during each window are the **recompute** reading its own products a page at a time - not the storefront's reads.

A shorter run on the same stack, at 100 connections with the sale created three seconds in, showed the same shape and a worst single request of **1 861 ms**. It is worth quoting because a p99 hides it and one caller really did wait nearly two seconds.

The answer to the question as asked

The case study asks for a strategy that prevents bottlenecks under simultaneous heavy reads and writes. Measured: ****the writes never block the reads, because the reads never reach the store the writes contend for.**** What the write storm costs is latency for as long as it runs - a quarter of the throughput, double the tail - and it costs nothing to availability.

A near-miss worth recording

One transaction-delta reading came back **negative** - before=260, after=11. PostgreSQL's cumulative counters had been reset underneath the measurement by other work on the same machine. The number was real, reproducible, and about nothing. It was caught because a negative delta is impossible, not because the method was sound. Later runs take an idle sample first, so the floor is visible and a reset shows up as an impossibility rather than as a plausible small number.

Vantage

Single machine, containers on one host, PostgreSQL 16, Redis 7, Accessories holding 100 000 products. Loopback with no network between client and server, so these latencies are a **ceiling for this machine** rather than a service level.

Each figure is one run, not a median of three, and the quiet and loaded runs were taken minutes apart on the same stack.

Edge cases

Reasoned from the mechanisms rather than from the code, then run live against the compose stack.

A promotion whose whole window is shorter than the sweep interval

The reconciler sweeps every five minutes. A promotion that opens and closes inside that window is invisible to it, so the delayed boundary jobs are the only thing that can apply and retire it.

A 60-second window, watched at five-second intervals from outside:

```
03:13:41 10000 no promotion    <- before the window
03:13:56 5000  "Blink ..."    <- 4 s after startsAt
03:14:57 10000 no promotion    <- 5 s after endsAt
```

****Both boundaries fired within five seconds of their instant, and the reconciler never touched it.**** The sweep is the repair for a lost announcement, not the mechanism, and this is the difference showing.

Promotion validation and conflict

Probe | Result

100 % percentage discount | refused 409 - another promotion already covered that target and window

zero-value discount | 400

promotion on a product that does not exist | 404 No such product

endsAt before startsAt | 400

a window entirely in the past | 400

percentage above 100 | 400

The 404 is worth naming: a foreign-key violation reaches the caller as "no such product" rather than as a 500 or a driver message.

Product validation

Probe | Result

duplicate SKU on create | 409 A product with this SKU already exists

negative base price | 400

Storefront validation

Refused with the envelope: an unknown query field, pageSize outside 1-100, sort other than effectivePrice, an order that is not asc/desc, a 257-character category, a negative or non-numeric page, and an offset past 10 000 (page=501 at pageSize=20 is the inclusive boundary; 502 is refused).
Path parameter: 0, -1, abc, 1.5 and 1 OR 1=1 all 400; a well-formed id that does not exist is 404. **A malformed id never reaches the bigint column.**

Ingestion

* CRLF line endings, a quoted comma inside a field, and no trailing newline all

parse correctly.

* **A duplicate SKU inside one file** is deduped before the upsert, because ON CONFLICT DO UPDATE cannot affect the same row twice in one statement. The last row wins. But rowsProcessed counts distinct SKUs rather than rows read, so three data rows report rowsProcessed: 2, rowsRejected: 0 and ****the vendor's arithmetic does not close****. Folding them into rejected would be a different wrong number - a row that lost to a later row was not rejected. A third counter is the honest fix. Recorded, not taken.

* **A category move through re-import** leaves the old listing: CatOne 1 -> 0, CatTwo 0 -> 1. There is no product update endpoint (PATCH is 404), so a category move is reachable only through ingestion.

Vantage

Single machine, containers on one host. Functional results; the only timing here is the boundary-job latency, observed at five-second polling resolution, so "4 s" and "5 s" are upper bounds rather than measurements.

Failure injection

Each of these was produced by breaking the running stack, not by simulating it.

Two promotions racing for one product

Eight concurrent promotion writes on the same product, fired together:

one 201 and seven 409, one active row, the storefront serving the winner.

The exclusion constraint decides it inside PostgreSQL, which is why eight simultaneous writers cannot produce two active promotions. No application lock is involved and none would help.

Redis restarted mid-recompute - pass

A 50 % sale on Outerwear's 100 002 products, Redis restarted three seconds in.

Across the window: **70 x 200, 13 x 503** - about six and a half seconds of honest unavailability saying the read model cannot be reached. ****No stale price and no fallback to PostgreSQL.**** The api and the consumer stayed up, and afterwards all 100 002 products carry the sale with the queue drained.

Unavailable rather than wrong, and self-correcting. That is what the storefront's 503 is for.

The reconciler's schedule survived a real Redis restart

The same restart hit the reconciler, which logged:

```
[ioredis] Unhandled error event: Error: connect ECONNREFUSED 172.23.0.3:6379  
{"since":"2026-09-14T00:43","windowEnd":"2026-09-14T01:43","repaired":2,"msg":"boundary sweep complete"}
```

The process stayed up and **the next sweep fired on schedule**. BullMQ upserts the next repeatable iteration when a worker picks a job up and, on failure, emits error and returns rather than throwing - so the five-minute chain could have died silently with the container still reading as healthy. It did not.

PostgreSQL restarted mid-recompute - the one defect

The recompute itself survived perfectly: BullMQ retried the pages whose worker had gone and **100 001 of 100 001 products came back** after recovery.

But the api process died and Docker restarted it. RestartCount went to 1 on both the api and the consumer, and what they printed on the way out was a raw terminating connection due to administrator command stack from pg-protocol, outside the logger.

createPool attached no error listener, and node-postgres documents that an idle client emitting an error with no listener takes the process down. So ****a database restart took the HTTP listener with it, and the storefront reads that never touch PostgreSQL failed for its duration**** - the design's central claim failing in the one place it can, through an unhandled event rather than through a query.

The read-model Redis client had the same gap without the fatality, printing outside

pino.

Fixed in PR #128, using each library's own documented remedy. Both tests emit the event and assert the log line rather than the survival, because a test asserting the process still runs passes whether or not the listener exists.

The two uniqueness consequences, confirmed

Neither is an endpoint defect; both are the missing vendor column.

* **One vendor's import silently replaces another's product.** `acme-corp` imported a SKU as a jacket at 105.00 with stock 5; `globex-inc` imported the same SKU as a parka at 262.50 with stock 99. **There is one row and it is Globex's.** Acme's name, price and stock are gone, with no error to either party and nothing in the response saying a replacement happened.

* **A third party can lock a vendor out permanently.** A file byte-identical to Acme's first one, sent by anyone, gets 409 already registered - the `file_sha256` index is global and nothing releases a hash. With no authentication, holding a competitor's catalogue file is enough. ADR-0005 records unique `(vendor, file_sha256)` as the fix for both and names the key's scope as the owner's to set.

Vantage

Single machine, containers on one host. Restart timings are as observed from outside; the 503 window is the count of refused requests during a load run rather than a stopwatch.

Drift repair and bulk retry, end to end

Run against a stack rebuilt from main **after** the feature merged, because a live probe proves the image and not the branch.

Drift repair

The boundary sweep repairs an announcement that was never published. This repairs something the queue cannot know about at all: a write that was published, consumed, and then lost.

The injection was real, not simulated. Five members were removed from the category:Accessories sorted set with ZREM, which is what an eviction or a partial restore leaves behind - no event was missed, so there is nothing for the sweep to re-announce.

```
before   PostgreSQL 100 000   Redis 100 000
ZREM x5   PostgreSQL 100 000   Redis  99 995
```

Two minutes later, unprompted, from the reconciler's own log:

```
boundary sweep complete   since 05:13:42   windowEnd 05:18:42   repaired 0
read model drift repaired category=Accessories products=100000 listed=99995 recomputed=100000
categories repaired by the drift check   repaired=1
```

ZCARD afterwards: **100 000**.

Three things that make this the right shape rather than merely a passing run:

* **The sweep found nothing and said so** (repaired 0) before the drift check found the loss. The two mechanisms answer different questions and the log shows both answering.

* **Only the affected category was rebuilt.** Accessories holds 100 000 of the catalogue's 500 330; nothing else was touched.

* **The line carries both counts**, so an operator reading it afterwards can tell how far apart the stores had drifted rather than only that something happened.

Bulk retry

npm run retry-failed -- <queue> against the live stack:

```
{"queue":"promotions","retried":0,"msg":"failed jobs returned to waiting"}
{"queues":["promotions","products","ingestion","maintenance"],"msg":"name one queue: ..."}

```

The second line is the refusal of a name that is not a queue, and the list comes from the routing map rather than from a second list in the script.

****The mechanism itself is proved in integration rather than here, and deliberately so.**** Manufacturing a failed set in a running stack means breaking a worker on purpose; the integration test does exactly that, with a consumer that rejects everything, against a real BullMQ.

That test is also what found the defect: **three jobs were retried 157 times** before the bound went in. A retried job that fails again re-enters the failed set while the command is still walking it, so a loop that stopped when the set emptied would never stop against a consumer that rejects everything. Each job is now

retried at most once per run, and that is the whole termination argument.

Vantage

Single machine, containers on one host, catalogue 500 330 products with 100 000 in Accessories. The drift injection and the repair are both from the live stack; the retry command's wiring is live and its bound is from integration against a real BullMQ.